

NOIP 模拟赛

NOIP

时间：2026 年 9 月 24 日 08:00 ~ 12:30

题目名称	无改编者	木琴修炼手册	暴怒豆包 vs 暴食蟑螂	双重承诺
题目类型	传统型	传统型	传统型	传统型
目录	change	feedback	furioso	promise
可执行文件名	change	feedback	furioso	promise
输入文件名	change.in	feedback.in	furioso.in	promise.in
输出文件名	change.out	feedback.out	furioso.out	promise.out
每个测试点时限	1.0 秒	2.0 秒	1.0 秒	2.5 秒
内存限制	512 MiB	512 MiB	512 MiB	512 MiB
测试点数目	20	20	20	20
测试点是否等分	否	否	否	否

提交源程序文件名

对于 C++ 语言	change.cpp	feedback.cpp	furioso.cpp	promise.cpp
-----------	------------	--------------	-------------	-------------

编译选项

对于 C++ 语言	- O2 - std=c++14 - static
-----------	---------------------------

注意事项（请仔细阅读）

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. `main` 函数的返回值类型必须是 `int`，程序正常结束时的返回值必须是 0。
3. 提交的程序代码文件的放置位置请参考各省的具体要求。
4. 因违反以上三点而出现的错误或问题，申诉时一律不予受理。
5. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
6. 选手提交的程序源文件必须不大于 100KB。
7. 程序可使用的栈空间内存限制与题目的内存限制一致。
8. 全国统一评测时采用的机器配置为：Intel(R) Core(TM) i7-8700K CPU @3.70GHz，内存 32GB。上述时限以此配置为准。
9. 只提供 Linux 格式附加样例文件。
10. 评测在当前最新公布的 NOI Linux 下进行，各语言的编译器版本以此为准。
11. 本场难度较易，请 AK 的同学不要大声喧哗。
12. 题目按照出题人主观难度从小到大排序。

无改编者 (change)

【题目背景】

百二八通过 114514 年的大学习，终于明白了英语翻译的最佳姿势，那就是完全忠实于原文，相同于原文顺序，这才是梦为完人，无改编者的完美翻译啊！

然而：

就让我成为你的风车，我想要成为你的风车。我仍然是你最大的风车。为什么一些被给予坏人的地位在瞬间于他们被发布于这个方法？

但是你是流程菜部门的伟大的金太阳植物学家，由于你上次进行过了 Relay Jump 大学习，所以你要用 No more Gazer 解决另一个问题。

【题目描述】

给定一个长度为 n 的整数序列 $a_1, a_2, \dots, a_n$ 。

你可以进行任意次操作，也可以不进行操作。每次操作如下：

1. 选择一个下标 i ，满足 $1 \leq i \leq n$ 。
2. 选择集合 $1, 2, \dots, i-1$ 或 $i+1, i+2, \dots, n$ 。
3. 对所选集合中的每个下标 j ，执行 $a_j \leftarrow 2a_i - a_j$ 。

你需要使最终序列 $a_1, a_2, \dots, a_n$ 单调不降，并且所有元素均为正数。在此条件下，求 a_n 的最小值。

由于你是伟大的金太阳植物学家，所以你要进行多次实验。本题共有 T 组数据。

注意：操作过程中，序列中的元素可以为负数或零。

【输入格式】

从 `change.in` 中读入数据。

第一行一个非负整数 T ，表示数据组数。

对于每组数据：

第一行一个正整数 n ，表示序列长度。

第二行 n 个整数 $a_1, a_2, \dots, a_n$ ，表示初始序列。

【输出格式】

输出到 `change.out`。

对于每组数据，输出一行一个整数，表示 a_n 的最小值。

可以证明，在给定的数据范围内，每组数据都可以通过有限次操作满足要求。

【输入输出样例 #1】

输入 #1

```
1 1
2 5
3 6 3 5 5 2
```

输出 #1

```
1 10
```

【说明/提示】

本题采用子任务捆绑测试。

对于下发的 `change1,2,3,4,5,6,7.in/ans` 样例，其分别满足子任务 2,3,3,5,6,7,7 的限制。

【数据范围】

对于所有数据，保证 $0 \leq T \leq 10^5$ ， $2 \leq n \leq 10^5$ ， $1 \leq a_i \leq 10^9$ 。单个测试点中，所有数据的 n 之和小于 10^6 。

子任务编号	$n \leq$	特殊性质	分值
1	10^5	A	1
2	$n = 2$	无	10
3	5	B	15
4	10^5	C	19
5	10^5	D	10
6	10^5	E	20
7	10^5	无	25

- 特殊性质 A： $T = 0$ 。
- 特殊性质 B： $T \leq 2$ ，且 $a_i \leq 5$ 。
- 特殊性质 C： 存在正整数 k ，使得对于所有 $1 \leq i \leq n$ ，均有 $a_i = ki$ 。
- 特殊性质 D： 初始序列 $a_1, a_2, \cdots, a_n$ 单调不降。
- 特殊性质 E： 保证不对 a_1 做任何修改也可以达到最优解。

木琴修炼手册 (feedback)

【题目背景】

“全琴界修为最高的人都卡在浮木境，从来没有人能将自己的木琴提升到滚木境。”

“我发现了突破浮木境的方法！”

“快说！”

“传说只要成为那项神秘的职业——出题人，再举办一场属于自己的模拟赛，就会有天地异象汇聚，万千选手的反馈化作全界精华，洗刷他的木琴，助其踏入传说中的滚木境。”

小 Z 若有所思地点了点头，缓缓起身。

“那就跟着我，良心恶心一个慢动作，恶心良心慢动作重播。”

“？你在唱什么。”

“这比赛，给你快乐，你有没有爱上我。”

三天后，小 Z 成为了一场模拟赛的出题人。

全琴界也第一次听见了木琴滚动的声音。

【题目描述】

小 Z 的模拟赛中有 n 道题，第 i 道题的权值为 a_i 。所有题目的权值互不相同。

小 Z 会将每道题设置为良心题或恶心题，并任意安排这 n 道题的顺序。

选手按照排列后的顺序依次完成这些题目。初始时，模拟赛的良心值与恶心值均为 0。

- 完成一道良心题后，良心值增加这道题的权值。

- 完成一道恶心题后，恶心值增加这道题的权值。

每完成一道题，选手都会根据当前良心值与恶心值的大小关系判断这场模拟赛：

- 若良心值严格大于恶心值，则认为这是良心场。

- 若恶心值严格大于良心值，则认为这是恶心场。

- 若两者相等，则不认为是良心场或恶心场。

小 Z 希望选手完成第 i 道题后，对这场模拟赛的看法为 s_i 。其中，字符 G 表示良心场，字符 B 表示恶心场。

但是，小 Z 不小心忘记了题目的顺序，也忘记了每道题是良心题还是恶心题，只记得这 n 道题的权值。

请你构造一种满足要求的题目顺序以及每道题的类型。

【输入格式】

从 `feedback.in` 中读入数据。

第一行一个正整数 n ，表示题目数量。

第二行 n 个互不相同的正整数 $a_1, a_2, \dots, a_n$ ，表示各道题的权值。

第三行一个长度为 n 的字符串 s 。其中， $s_i \in \text{G, B}$ 。

【输出格式】

输出到 `feedback.out`。

输出 n 行。第 i 行输出一个正整数和一个字符，分别表示排列后第 i 道题的权值，以及这道题是良心题还是恶心题。

若这道题是良心题，输出字符 `G`；否则输出字符 `B`。

给定的每个权值必须且只能使用一次。若存在多种合法方案，输出任意一种即可。

【输入输出样例 #1】

输入 #1

15

23 8 2 13 7

3GGBGG

输出 #1

13 G

22 B

38 B

413 G

57 G

【说明/提示】

本题采用特殊评测与子任务捆绑测试。

可以证明，对于所有满足数据范围的输入，至少存在一种合法方案。

对于下发的 `feedback1,2,3,4,5,6.in/ans` 样例，其分别满足子任务 1,2,3,4,5,6 的限制。

【数据范围】

对于所有数据，保证 $1 \leq n \leq 10^6$ ， $1 \leq a_i \leq 10^9$ ，所有 a_i 互不相同，且 $s_i \in \text{G,B}$ 。

子任务编号	$n \leq$	特殊性质	分值
1	8	无	10
2	10^6	A	15
3	10^6	B	15
4	10^6	C	15
5	10^6	D	20
6	10^6	无	25

特殊性质 A： s 中没有字符 `B`。

特殊性质 B: s 中恰好有一个字符 **B**。

特殊性质 C: 对于所有 $1 \leq i \leq n$, 均有 $a_i = i + n$ 。

特殊性质 D: s 中的字符 **G** 和 **B** 交替出现。

暴怒豆包 vs 暴食蟑螂 (furioso)

【题目背景】

在某一个平行世界.....

豆包是五根失败的草中的一根，还是左手性草的第五根。（不应该是第四根吗？）

一天，一只蟑螂来吃草，会发生什么呢？

这里本来有个 T1，但是我无疑是暴怒的。

所以你需要完成这个 T3。

【题目描述】

给定一个正整数 n 和一个长度为 n 的正整数序列 $a_1, a_2, \dots, a_n$ 。

从位置 i 跳到位置 j ，其中 $1 \leq i < j \leq n$ ，需要花费 $\min(a_i, a_{i+1}, \dots, a_j) \times (j - i)^2$ 的代价。一次跳跃只能跳向编号更大的位置，多次跳跃的总代价为每次跳跃的代价之和。

对于每个 $k = 1, 2, \dots, n$ ，求从位置 1 经过若干次跳跃到达位置 k 的最小总代价。

当 $k = 1$ 时，可以不进行任何跳跃，因此答案为 0。

【输入格式】

从 `furioso.in` 中读入数据。

第一行一个正整数 n 。

第二行 n 个正整数 $a_1, a_2, \dots, a_n$ 。

【输出格式】

输出到 `furioso.out`。

输出一行 n 个整数，其中第 k 个整数表示从位置 1 到达位置 k 的最小总代价。

相邻两个整数之间用一个空格分隔。

【输入输出样例 #1】

输入 #1

```
1 4
2 1 4 4 4
```

输出 #1

```
1 0 1 4 8
```

【输入输出样例 #2】

输入 #2

```
1 3
2 2 1 3
```

输出 #2

```
1 0 1 2
```

【输入输出样例 #3】

输入 #3

```
1 6
2 1 4 1 6 3 2
```

输出 #3

```
1 0 1 2 3 6 8
```

【输入输出样例 #4】

输入 #4

```
1 2
2 1 2
```

输出 #4

```
1 0 1
```

【说明/提示】

本题采用子任务捆绑测试。

下发的大样例 `furioso1,2,3,4,5,6,7,8.in/ans` 分别满足子任务 1,1,2,3,4,5,6,6 的限制。

【数据范围】

对于所有数据，保证 $1 \leq n \leq 4 \times 10^5$ ， $1 \leq a_i \leq n$ 。

子任务编号	$n \leq$	特殊性质	分值
1	10	无	1
2	5×10^3	无	14
3	4×10^5	A	10
4	4×10^5	B	20
5	4×10^5	C	10
6	4×10^5	无	45

特殊性质 A: $n \geq 10$, 且对于所有 $1 \leq i \leq n$, 均有 $a_i \leq 10$ 。
特殊性质 B: 对于所有 $1 \leq i \leq n$, 均有 $a_i \geq 4 \times 10^4$ 。
特殊性质 C: 序列 $a_1, a_2, \cdots, a_n$ 单调不降。

双重承诺 (promise)

【题目背景】

小 Z 在宇宙听写大会中赢得了冠军！

由于奖金非常丰厚，所以小 Z 提前过上了退休生活。但是小 Z 认为自己不能坐吃山空，所以他用奖金开了一家工厂赚钱。

【题目描述】

小 Z 的工厂接到了 n 份订单，编号分别为 $1, 2, \dots, n$ 。第 i 份订单的加工时间为 a_i 。

工厂原本承诺按照编号从小到大的顺序完成所有订单。也就是说，第 i 份订单将第 i 个完成，其承诺完成时间为

$$\sum_{j=1}^i a_j。$$

现在，工厂可以任意调整订单的加工顺序。加工从时刻 0 开始，每次加工一份订单，中途不能暂停，也不能空闲。

若第 i 份订单实际第 p_i 个完成，实际完成时间为 t_i ，则需要支付的赔偿金为：
$$\max\left(t_i - \sum_{j=1}^i a_j, 0\right) + a_i \times \max(p_i - i, 0)。$$

也就是说，完成时间每推迟一个单位，需要赔偿 1 元；完成次序每推迟一位，需要赔偿 a_i 元。两项赔偿相加，得到这份订单的总赔偿金。

此外，对于一次询问，工厂需要选择恰好 R 份订单进行特别检验，并将这些订单安排在加工顺序的前 R 个位置。前 R 个位置中的订单必须恰好为选中的订单，但它们之间的加工顺序可以任意安排。

选中第 i 份订单需要支付 $A \times a_i$ 元检验费。检验不占用加工时间。

给定 q 次询问，每次给出一个 R 。对于每次询问，求检验费与所有订单赔偿金之和的最小值，各次询问相互独立。

【输入格式】

从 `promise.in` 中读入数据。

第一行三个整数 n, q, A ，分别表示订单数量、询问数量和检验费用系数。

第二行 n 个整数 $a_1, a_2, \dots, a_n$ ，表示每份订单的加工时间。

第三行 q 个整数 $R_1, R_2, \dots, R_q$ ，其中 R_i 表示第 i 次询问需要特别检验的订单数量。

【输出格式】

输出到 `promise.out`。

输出 q 行，第 i 行输出一个整数，表示第 i 次询问的答案。

【输入输出样例 #1】

输入 #1

```
1 4 1 6
2 4 1 2 3
3 2
```

输出 #1

```
1 29
```

【输入输出样例 #2】

输入 #2

```
1 6 1 2
2 1 2 8 1 2 5
3 3
```

输出 #2

```
1 17
```

【说明/提示】

本题采用子任务捆绑测试。

对于下发的 `promise1,2,3,4,5,6,7.in/ans` 样例，其分别满足子任务 1,2,3,4,5,6,7 的限制。

【样例解释 1】

选择第 2,3 份订单进行特别检验，并按照 2,3,1,4 的顺序加工，检验费用为 $(1 + 2) \times 6 = 18$ 。

第 1 份订单在时刻 7 完成，比承诺完成时间 4 晚 3 个单位时间。同时，它是第 3 个完成的订单，比承诺的次序靠后 2 位。

因此，第 1 份订单的赔偿金为 $3 + 2 \times 4 = 11$ ，其余订单均不需要赔偿，故总费用为 $18 + 11 = 29$ 。可以证明，不存在费用更小的方案。

【样例解释 2】

选择第 1,2,4 份订单进行特别检验，并按照 1,2,4,3,5,6 的顺序加工，检验费用为 $(1 + 2 + 1) \times 2 = 8$ 。

第 3 份订单在时刻 12 完成，比承诺完成时间 11 晚 1 个单位时间。同时，它是第 4 个完成的订单，比承诺的次序靠后 1 位。

因此，第 3 份订单的赔偿金为 $1 + 1 \times 8 = 9$ ，其余订单均不需要赔偿，故总费用为 $8 + 9 = 17$ 。可以证明，不存在费用更小的方案。

【数据范围】

对于所有测试数据，保证 $1 \leq n, q \leq 1 \times 10^5$ ， $1 \leq A, a_i \leq 10^6$ ， $1 \leq R_i \leq n$ 。

子任务编号	n	特殊性质	分值
1	≤ 10	无	10
2	≤ 20	无	10
3	$\leq 4 \times 10^3$	无	15
4	$\leq 1 \times 10^5$	A	5
5	$\leq 1 \times 10^5$	B	15
6	$\leq 4 \times 10^4$	无	20
7	$\leq 1 \times 10^5$	无	25

特殊性质 A：对于所有 $1 \leq i < n$ ，均有 $a_i = a_{i+1}$ 。

特殊性质 B：序列 a 中至多存在两种不同的数。